

기술 리포트 (Technical Report)

도서 리뷰 3-class 감성 분석

웹 크롤링 기반 데이터 수집 · 전처리 · BiLSTM 모델 학습, 그리고 도서 인사이트 서비스 구축

- 주제 : 도서 리뷰 텍스트의 긍정 / 중립 / 부정 자동 분류
- 데이터 : YES24 · Goodreads · Watcha 도서 리뷰 (웹 크롤링)
- 모델 : Embedding → 양방향 LSTM(BiLSTM) → Dense Softmax
- 산출물 : 학습 노트북, 감성 분석 모델, Streamlit 웹 서비스

작성자 : 김채우 (학번 2503110166)

작성일 : 2026년 6월

목차

1. 프로젝트 개요
2. 데이터 수집 (웹 크롤링)
3. 데이터 전처리
4. 모델 설계와 학습
5. 학습 결과와 평가
6. 개발 과정의 문제점과 해결
7. 모델의 한계
8. 향후 개선 계획
9. Streamlit 웹앱 서비스
10. 맺음말

1. 프로젝트 개요

본 프로젝트의 목표는 온라인 서점·리뷰 플랫폼에 흩어져 있는 도서 리뷰 텍스트를 대규모로 수집하고, 각 리뷰가 **긍정 · 중립 · 부정** 중 어떤 감정을 담고 있는지 자동으로 분류하는 한국어 감성 분석 모델을 만드는 것이다. 나아가 모델의 예측 결과를 책 단위로 집계하여, 한 권의 책이 독자들에게 어떻게 평가받는지(평점·호불호·장단점)를 한눈에 보여 주는 **도서 인사이트 웹 서비스**까지 구축하였다.

전체 파이프라인은 다음과 같이 다섯 단계로 구성된다.

- ① **데이터 수집** : Selenium·API 크롤러로 YES24·Goodreads·Watcha 세 곳에서 리뷰를 수집한다.
- ② **전처리** : 평점을 3-class 라벨로 변환하고, 영어 리뷰는 한국어로 번역한 뒤, 텍스트 정제와 형태소 분석을 거쳐 정수 시퀀스로 만든다.
- ③ **모델 학습** : 양방향 LSTM 기반 분류기를 클래스 가중치와 함께 학습한다.
- ④ **예측·집계** : 학습한 모델로 세 사이트 리뷰 전체의 감성을 예측하고, 책 단위로 통계를 사전 계산한다.
- ⑤ **서비스화** : Streamlit으로 검색·랭킹·트렌드·비교·실시간 분석을 제공하는 웹앱을 만든다.

이 리포트는 위 단계를 따라가며 **어떤 데이터를 어떻게 모았고, 어떤 전처리와 학습을 거쳤으며**, 그 과정에서 **어떤 문제에 부딪혀 어떻게 해결했는지**를 구체적으로 기록한다. 이어서 완성된 모델이 가진 한계와 이를 보완하기 위한 향후 계획, 마지막으로 직접 만든 웹 서비스의 구조와 기능을 설명한다.

2. 데이터 수집 (웹 크롤링)

학습과 예측에 쓸 도서 리뷰를 확보하기 위해 성격이 서로 다른 세 개의 사이트를 크롤링하였다. 세 사이트는 언어(한국어·영어), 평점 제공 여부, 수집 방식(HTML 파싱·내부 API)이 모두 달라서 각각 별도의 크롤러를 작성했다. 수집 결과를 요약하면 다음과 같다.

사이트	언어	평점	수집량	역할
YES24	한국어	있음 (1~10)	리뷰 약 42,890건	학습 + 예측
Goodreads	영어	있음 (1~5)	리뷰 약 270,414건	번역 후 학습 + 예측
Watcha	한국어	없음	코멘트 약 105,185건	예측 전용

표 1. 수집 대상 세 사이트의 특성과 역할. 평점이 있는 YES24·Goodreads는 라벨을 만들 수 있어 학습에 쓰고, 평점이 없는 Watcha는 학습된 모델로 감성을 예측하는 용도로만 사용했다.

2.1 YES24 크롤러 — 한국어 + 평점 (학습의 핵심 축)

YES24는 한국어 리뷰에 1~10점 평점이 함께 달려 있어, 별도 라벨링 없이 평점만으로 감성 라벨을 만들 수 있는 가장 중요한 데이터원이다. 크롤러(**yes24_bestseller_crawler.py**)는 Selenium 헤드리스 Chrome으로 동작하며, 다음과 같은 전략으로 리뷰를 광범위하게 모았다.

- **카테고리 전면 확장** : 소설·경제경영·인문·과학·IT 등 **80개 이상**의 세부 카테고리를 코드로 정의해 특정 장르에 치우치지 않도록 했다.
- **4종 리스팅 모드** : 스테디셀러·실시간 베스트·주간 베스트·신간(steadyseller, realtimebestseller, bestseller, newbook)을 순회하여 인기·신간 도서를 폭넓게 확보했다.
- **2단계 수집(Phase)** : 1단계로 카테고리 리스팅을 훑고, 2단계로 연도·월별 베스트셀러를 추가로 수집해 과거 도서까지 포함했다.
- **페이지당 수집량 향상** : 한 페이지에 보여 주는 도서 수를 24 → 120으로 키워 같은 시간에 더 많이 모았다.

- **리뷰 본문·평점·작성일 파싱** : 리뷰 평점은 별점 아이콘의 CSS 클래스(total_rating_N)에서 정수로 추출하고, 리뷰 영역을 페이지네이션 버튼으로 끝까지 넘기며 수집했다.

2.2 Goodreads 크롤러 — 영어 대규모 데이터

YES24만으로는 학습에 필요한 **부정·중립** 리뷰가 크게 부족했기 때문에(2.1과 6장 참조), 전 세계 최대 독서 커뮤니티인 Goodreads에서 영어 리뷰를 대규모로 확보했다. 크롤러(`goodreads_crawler.py`)는 다음 두 종류의 목록을 펼쳐 수만 권 규모의 수집 큐를 만든다.

- **Listopia 인기 리스트** : 'Best Books Ever', 'Best Mystery/Thriller', 'Best Self Help Books' 등 약 20개의 인기 리스트(리스트당 약 100권).
- **장르·주제 셸프(shelf) 태그** : fiction·mystery·science-fiction·romance·history·self-help 등 100개 이상의 태그를 페이지 단위로 펼쳐 수집.
- **'발견 즉시 크롤링' 방식** : 목록을 모두 펼친 뒤 시작하는 대신, 책을 발견하는 즉시 리뷰를 수집해 URL이 아무리 많아도 시작이 지연되지 않도록 설계했다.

리뷰는 평점(1~5)·작성자·작성일·본문·좋아요 수를 dataclass로 구조화해 저장했으며, 최종적으로 약 27만 건(reviews.csv)을 확보했다.

2.3 Watcha 크롤러 — 내부 API 기반 (예측 전용)

Watcha(왓차피디아)는 별점이 코멘트와 분리되어 있어 학습용 라벨로 쓰기 어려웠지만, 한국어 코멘트가 풍부해 **예측 대상**으로 가치가 컸다. 크롤러(`watcha_books_crawler.py`)는 HTML을 긁는 대신 로그인 세션으로 쿠키를 확보한 뒤 사이트 **내부 API**를 직접 호출하는 방식을 택해 안정적이고 빠르게 수집했다.

- **책 덱(deck) 기반 탐색** : 사용자들이 만든 도서 컬렉션(덱) URL 목록을 시드로 삼아 도서를 모으고, 각 도서의 코멘트를 API로 페이지 단위로 가져왔다.
- **품질 필터링** : 학습·분석에 부적합한 코멘트를 수집 단계에서 걸러냈다. 글자 수 15자 미만/5000자 초과, URL이 포함된 코멘트, 날짜만 적힌 메모성 코멘트를 제외했다.
- **요청 속도 제어** : 같은 계정 쿠키를 공유하므로 워커 1개로 운영하고, 요청 사이에 0.4~0.7초 지연을 두며 30회 요청마다 15초씩 선제적으로 쉬어 429(Too Many Requests) 차단을 예방했다.

그 결과 코멘트가 10개 이상 달린 책을 중심으로 약 10.5만 건의 한국어 코멘트를 확보했다.

2.4 세 크롤러의 공통 안정화 장치

수만~수십만 건을 장시간에 걸쳐 수집하려면 중간에 멈추거나 차단당해도 데이터를 잃지 않아야 한다. 세 크롤러는 공통적으로 다음 장치를 갖췄다.

- **이어하기(resume)** : 기존 CSV를 읽어 이미 수집한 책·리뷰를 건너뛰고 중단 지점부터 이어서 수집한다.
- **중복 제거** : (책 URL, 리뷰 앞부분) 조합을 키로 같은 리뷰의 중복 저장을 막고, 종료 시 한 번 더 dedup한다.
- **부분 저장** : 책 단위로 처리할 때마다 CSV를 갱신해, 예기치 못한 종료에도 직전까지의 결과가 남도록 했다.
- **차단 회피** : 무작위 대기 시간(2~5초), 실제 브라우저처럼 보이는 User-Agent, 자동화 탐지 우회 옵션, 세션 만료 시 드라이버 자동 재시작 등을 적용했다.

3. 데이터 전처리

수집한 원시 리뷰를 모델이 학습할 수 있는 정수 시퀀스로 바꾸기까지는 라벨링 → 번역 → 정제 → 형태소 분석 → 인코딩의 다섯 단계를 거친다. 학습 노트북(sentiment_analysis_3class.ipynb)에 전 과정이 구현되어 있다.

3.1 평점 → 3-class 라벨 변환

평점이 있는 두 사이트의 점수 체계가 달라(YES24 1~10, Goodreads 1~5), 각각의 척도에 맞춰 부정·중립·긍정으로 구간을 나누었다.

- **YES24(1~10)** : 1~4점 → 부정, 5~7점 → 중립, 8~10점 → 긍정
- **Goodreads(1~5)** : 1~2점 → 부정, 3점 → 중립, 4~5점 → 긍정

이렇게 라벨을 만든 결과, 두 데이터의 분포는 극명하게 달랐다. YES24는 긍정이 압도적으로 많았던 반면(긍정 약 39,841건 / 중립 2,303건 / 부정 736건), Goodreads는 세 클래스가 비교적 고르게 분포했다(긍정 134,852 / 중립 59,707 / 부정 72,137). 이 **심한 불균형**이 이후 전처리·학습 설계의 핵심 변수가 되었다(6장 참조).

3.2 영어 리뷰 번역 (English → Korean)

한국어 모델을 만드는 것이 목표이므로 Goodreads 영어 리뷰는 한국어로 번역해 학습에 합류시켰다. 클라우드 번역 API는 비용·속도 제약이 커서, 로컬에서 동작하는 신경망 번역기 **Helsinki-NLP opus-mt-tc-big-en-ko**를 사용했다. 핵심 설계는 다음과 같다.

- **클래스 균형 표본 추출** : 27만 건을 모두 번역하지 않고, 클래스별 최대 10만 건만 뽑아 번역 대상을 약 231,844건(부정 72,137 + 중립 59,707 + 긍정 100,000)으로 구성했다. 긍정 쓸림을 줄이려는 의도였다.
- **배치 번역** : 32문장씩 묶어 GPU로 번역해 처리량을 높였고, 너무 긴 입력은 1,500자로 잘라 메모리 초과를 방지했다.
- **실패 격리** : 배치 번역이 실패하면 해당 배치를 문장 단위로 다시 시도해, 한 문장의 오류가 배치 전체를 날리지 않게 했다.
- **결과 캐싱** : 번역 결과를 CSV로 저장해, 재실행 시 번역 단계를 통째로 건너뛸 수 있게 했다.

※ 번역은 단일 GPU로 약 23시간이 소요된 가장 비용이 큰 단계였다. 캐싱이 없었다면 이후 실험을 반복할 때마다 이 시간을 다시 치러야 했다.

3.3 텍스트 정제

번역된 한국어 리뷰와 YES24 리뷰를 하나로 합친 뒤(약 254,944건), 정규식으로 **한글과 공백만 남기고** 숫자·영문·이모지·특수문자를 모두 제거했다. 이어 연속 공백을 하나로 줄이고, 정제 후 내용이 비어 버린 리뷰와 완전히 중복되는 리뷰를 삭제했다. 그 결과 학습 데이터는 **213,432건**(부정 56,150 / 중립 47,071 / 긍정 110,211)으로 정리되었다.

3.4 형태소 분석과 토큰화

한국어는 조사·어미가 붙어 띄어쓰기만으로는 의미 단위를 나누기 어렵다. 따라서 KoNLPy의 **Okt 형태소 분석기**로 각 리뷰를 형태소 단위로 쪼갰다(예: '정말 강추합니다' → '정말' · '강추' · '합니다'). 21만여 건의 형태소 분석에는 약 19분이 걸렸으며, 이 결과 역시 CSV로 캐싱해 재학습 시 재사용하도록 했다.

3.5 학습/테스트 분리 · 정수 인코딩 · 패딩

전처리된 데이터를 다음 절차로 모델 입력 형태로 변환했다.

- **증화 분할** : 클래스 비율을 유지한 채 9 : 1로 나눠 학습 192,088건 / 테스트 21,344건을 만들었다.
- **정수 인코딩** : 빈도 상위 **5만 개** 단어로 단어 사전을 만들고(전체 고유 단어는 144,814개), 각 형태소를 정수 인덱스로 바꿨다. 사전에 없는 단어는 OOV 토큰으로 처리한다.
- **패딩** : 리뷰 길이를 100 토큰으로 통일(MAX_LEN=100)했다. 짧으면 0으로 채우고, 길면 잘랐다.

주의 : 단어 사전은 데이터 누수를 막기 위해 **학습 데이터에만** 학습시켰다(테스트 데이터로 사전을 만들지 않음)

3.6 전처리 단계별 데이터 건수 변화

단계	건수	비고
라벨링 가능 원본	약 309,576건	YES24 42,880 + Goodreads 266,696
번역 대상(클래스 균형)	231,844건	긍정만 10만으로 상한
YES24 + 번역 통합	254,944건	번역 실패·빈 값 제거 후
정제 · 중복 제거	213,432건	한글만 남기고 중복 삭제
학습 / 테스트	192,088 / 21,344	증화 9:1 분할

표 2. 전처리를 거치며 데이터가 걸러지는 과정. 통합 단계에서 약 1.9만 건은 번역 결과가 비어 제외되었고, 정제·중복 제거에서 약 4만 건이 추가로 줄었다.

4. 모델 설계와 학습

4.1 모델 구조

순서가 중요한 텍스트의 특성을 살리기 위해 **양방향 LSTM(BiLSTM)**을 핵심으로 하는 순환 신경망을 사용했다. 문장을 앞에서 뒤로, 뒤에서 앞으로 동시에 읽어 문맥을 양방향으로 파악한다. 전체 구조는 다음과 같다.

- **Embedding** (단어 사전 50,001 → 128차원) : 각 단어를 128차원 벡터로 표현
- **Bidirectional LSTM** (유닛 64) : 문장을 양방향으로 읽어 문맥 벡터 생성
- **Dropout 0.4 → Dense 64 (ReLU) → Dropout 0.3** : 과적합 억제 + 비선형 변환
- **Dense 3 (Softmax)** : 부정·중립·긍정 세 클래스의 확률 출력

손실 함수는 categorical_crossentropy, 옵티마이저는 Adam(학습률 0.001)을 사용했다.

4.2 학습 설정

- **클래스 가중치(class_weight)** : 데이터가 적은 클래스에 더 큰 가중치를 부여했다(부정 1.27 / 중립 1.51 / 긍정 0.65). 긍정 쓸림을 모델이 그대로 학습하지 않도록 보정한 것이다.
- **조기 종료(EarlyStopping)** : 검증 손실이 3 에폭 동안 나아지지 않으면 멈추고, 가장 좋았던 가중치로 되돌리도록 했다.
- **체크포인트(ModelCheckpoint)** : 검증 손실이 가장 낮은 시점의 모델을 자동 저장했다.
- **배치 크기 256, 최대 15 에폭, 검증 분할 10%**로 학습을 진행했다.

5. 학습 결과와 평가

학습은 조기 종료에 의해 5 에폭에서 멈췄고, 검증 손실이 가장 낮았던 **2 에폭**의 가중치가 최종 모델로 선택되었다. 학습 정확도는 에폭이 지날수록 계속 올랐지만(약 0.49 → 0.62) 검증 손실은 2 에폭

이후 오히려 나빠져, 모델이 빠르게 과적합되고 있음을 보여 주었다.

5.1 테스트 성능

따로 떼어 둔 테스트 21,344건에 대한 최종 성능은 **정확도 57.2%**였다. 클래스별 지표는 다음과 같다.

클래스	정밀도	재현율	F1	테스트 건수
부정	0.473	0.548	0.508	5,615
중립	0.371	0.492	0.423	4,707
긍정	0.793	0.618	0.695	11,022
전체 정확도	—	—	0.572	21,344

표 3. 클래스별 분류 성능. 긍정은 비교적 잘 맞혔으나(F1 0.70), 중립은 가장 어려웠다(F1 0.42).

혼동 행렬을 보면 오류의 성격이 더 분명해진다. 실제 **중립** 리뷰는 부정(1,420건)·긍정(969건)으로 폭 넓게 새어 나갔고, 실제 **긍정** 리뷰의 상당수도 중립(2,202건)·부정(2,006건)으로 잘못 분류되었다. 즉 모델은 **긍정과 부정의 양 극단은 어느 정도 잡아내지만, 중간 지대(중립)를 거의 구분하지 못한다**.

5.2 예측 결과의 타당성 점검

헤드라인 정확도(57%)는 낮지만, 모델이 무의미하게 찍는 수준은 아니라는 근거도 있다. 학습에 쓰지 않은 평점과 예측을 대조해 보면, YES24 리뷰에서 모델이 **긍정**으로 예측한 리뷰의 실제 평균 평점은 9.49점, **부정**으로 예측한 리뷰는 3.34점, **중립**은 5.94점이었다. 예측 감성이 실제 평점 순서와 또렷하게 일치한다. 이는 개별 리뷰 단위 정확도는 낮아도, **책 단위로 수백~수천 건을 모아 보면 통계적으로 신뢰할 만한 신호**가 나온다는 뜻이며, 9장의 서비스 설계 근거가 된다.

6. 개발 과정의 문제점과 해결

프로젝트 전 과정에서 부딪힌 주요 문제와, 각각을 어떻게 해결했는지를 정리한다.

문제 1. YES24 데이터의 극심한 라벨 불균형

문제 — 가장 신뢰도 높은 한국어 데이터인 YES24가 긍정 92.9%(39,841 / 42,880)로 심하게 치우쳐 있었다. 리뷰를 자발적으로 남기는 독자는 대체로 좋게 평가하는 경향이 있어, 부정은 736건, 중립은 2,303건에 불과했다. 이대로 학습하면 모델은 '무조건 긍정'만 외쳐도 높은 정확도가 나오는 무의미한 분류기가 된다.

해결 — 부족한 부정·중립 표본을 채우기 위해 영어 Goodreads 데이터를 대규모로 추가 수집해 번역·합류시켰다. 그 결과 학습 데이터의 부정·중립 비중이 각각 26%, 22% 수준으로 올라가 세 클래스가 학습 가능한 균형에 도달했다.

문제 2. 영어 데이터의 번역 비용

문제 — 영어 리뷰를 합류시키려면 번역이 필수인데, 27만 건을 클라우드 API로 번역하면 비용과 시간이 감당되지 않았다. 또한 일부 리뷰는 512토큰을 초과할 만큼 길어 번역기에서 인덱싱 오류·메모리 초과를 일으켰다.

해결 — ① 로컬 신경망 번역기(Helsinki-NLP)를 써서 비용을 없앴다. ② 모든 데이터를 번역하는 대신 클래스별 최대 10만 건만 균형 표본으로 추출해 번역량을 줄였다(동시에 문제 1의 균형도 개선). ③ 긴 문장은 1,500자로 자르고, 배치 실패 시 문장 단위로 재시도하는 격리 로직을 두었다. ④ 번역·형태소 분석처럼 오래 걸리는 단계의 결과를 모두 CSV로 캐싱해, 이후 실험에서 재실행 시간을 없앴다.

문제 3. 학습 단계의 클래스 불균형

문제 — 데이터를 보강한 뒤에도 긍정(110,211)이 여전히 부정·중립의 두 배 규모였다. 다수 클래스에 끌려가면 소수 클래스의 재현율이 무너진다.

해결 — 'balanced' 방식으로 클래스 가중치를 계산해(부정 1.27 / 중립 1.51 / 긍정 0.65) 손실에 반영했다. 데이터가 적은 클래스의 오답에 더 큰 페널티를 주어, 모델이 소수 클래스도 학습하도록 유도했다.

문제 4. 빠른 과적합

문제 — 학습 정확도는 계속 올랐지만 검증 손실은 2 에폭 이후 오히려 증가했다. 모델이 일반화 대신 학습 데이터를 외우기 시작한 것이다.

해결 — Dropout(0.4 / 0.3)으로 정규화하고, EarlyStopping으로 검증 손실이 최저였던 2 에폭의 가중치를 복원해 최종 모델로 삼았다. 즉 '더 학습하면 나빠지는' 지점에서 멈춰 과적합된 모델이 배포되는 것을 막았다.

문제 5. 대용량·장시간 파이프라인의 안정성

문제 — 크롤링은 수십 시간, 번역은 약 23시간, 형태소 분석은 약 19분이 걸리는 등 단계마다 비용이 컸고, 중간에 한 번 끊기면 처음부터 다시 해야 할 위험이 있었다.

해결 — 크롤러에는 이어하기·부분 저장·중복 제거를, 노트북에는 단계별 캐시(번역 캐시·토큰화 캐시)를 도입했다. 덕분에 어느 단계에서 멈추더라도 그 지점부터 재개할 수 있어 반복 실험이 가능해졌다.

문제 6. 형태소 분석기의 Java 의존성

문제 — KoNLPy의 Okt 분석기는 내부적으로 Java(JDK)에 의존해, 환경이 갖춰지지 않으면 전처리 자체가 동작하지 않았다.

해결 — 학습·서비스 환경에 JDK를 설치하고, 클라우드 배포용으로는 시스템 패키지 설치 파일(packages.txt에 default-jdk)을 함께 제공해 배포 환경에서도 자동으로 Java가 깔리도록 했다.

7. 모델의 한계

문제들을 해결하며 동작하는 모델을 완성했지만, 본 모델은 다음과 같은 분명한 한계를 가진다. 이를 솔직히 기록하는 것이 다음 개선의 출발점이라고 보았다.

- **중간 수준의 정확도(57%)** — 특히 중립 클래스의 F1이 0.42로, 세 감정을 또렷이 가르기에는 부족하다. '그냥 그래요'처럼 미묘한 리뷰는 부정/중립을 자주 혼동한다.
- **번역에서 온 노이즈** — 학습 데이터의 다수가 기계 번역문이다. 번역 과정에서 어감·뉘앙스가 뭉개지고, 원어민이 쓴 자연스러운 한국어 리뷰와 문체가 달라(번역투) 실제 한국어 입력에 대한 일반화가 약하다. 실제로 번역 데이터의 평점 라벨과 모델 예측의 일치도는 한국어 데이터보다 눈에 띄게 낮았다.
- **평점 기반 라벨의 노이즈** — '평점 = 감정'이라는 가정은 항상 옳지 않다. 별 3개를 주면서도 호평을 쓰거나, 높은 점수에 불만을 덧붙이는 리뷰가 많아 라벨 자체에 잡음이 섞여 있다. 특히 중립 구간이 모호하다.
- **한국어 전용** — 한글만 남기는 정제 때문에 영어를 그대로 입력하면 분석되지 않으며, 한글이 전혀 없으면 기본값 '중립'으로 처리된다.

- **정제 과정의 정보 손실** — 숫자·영문·이모지·느낌표('!!!', 'ㅋㅋ') 같은 자모 표현을 모두 제거한다. 그러나 이런 요소(예: '!!!', 'ㅋㅋ', 'ㅠㅠ')는 감정의 강한 단서여서, 이를 버리면서 신호가 함께 사라진다.
- **사전학습 없이 처음부터 학습한 임베딩** — 대규모 말뭉치로 사전학습된 언어 모델을 쓰지 않고 임베딩을 무작위에서 학습했기에, 단어의 의미·문맥 이해력이 근본적으로 제한된다. 사전(5만 단어)에 없는 신조어·고유명사는 OOV로 처리되어 정보가 사라진다.
- **검증 불가능한 예측 대상** — Watcha 코멘트는 평점이 없어, 그에 대한 예측의 정확도를 직접 검증할 방법이 없다.

8. 향후 개선 계획

위 한계를 보완하기 위한 개선 방향을 우선순위 순으로 제시한다.

- ① **사전학습 한국어 언어 모델로 교체** — KLUE-BERT-KoELECTRA 같은 사전학습 모델을 미세조정(fine-tuning)하면, 문맥 이해력이 근본적으로 향상되어 가장 큰 성능 개선을 기대할 수 있다. 처음부터 학습한 BiLSTM의 임베딩 한계를 직접 해소하는 방법이다.
- ② **번역 의존도 축소** — 기계 번역문 대신 네이티브 한국어 부정·중립 리뷰를 직접 더 모은다. 알라딘·네이버책 등 다른 한국어 서점의 저평점 리뷰를 추가 크롤링하면 번역투 노이즈 없이 균형을 맞출 수 있다.
- ③ **라벨 품질 개선** — 평점만이 아니라 리뷰 본문의 신호를 함께 보는 라벨링, 모호한 중립 구간 재정의, 또는 우선 긍정·부정 2-class로 단순화한 뒤 중립을 점진적으로 도입하는 단계적 접근을 검토한다.
- ④ **전처리에서 감정 단서 보존** — 이모지·반복 문자('!!!', 'ㅠㅠ', 'ㅋㅋ')·강조 표현을 무작정 지우지 말고 특수 토큰으로 변환해 감정 신호로 활용한다.
- ⑤ **불균형·손실 함수 보강** — 클래스 가중치에 더해 Focal Loss나 오버/언더샘플링을 실험하고, 하이퍼파라미터(임베딩 차원·LSTM 유닛·드롭아웃)를 체계적으로 탐색한다.
- ⑥ **신뢰도 기반 보류(abstain)** — 예측 확률이 낮을 때는 '판단 보류'로 처리해, 자신 없는 예측을 강제로 한 클래스에 넣지 않도록 한다.
- ⑦ **예측 검증 체계** — 일부 표본을 사람이 직접 라벨링해 Watcha처럼 평점 없는 데이터의 예측 품질을 정기적으로 점검한다.

9. Streamlit 웹앱 서비스 — 도서 리뷰 인사이트

학습한 모델이 개별 리뷰 단위로는 정확도가 낮아도 **책 단위로 모으면 신뢰할 만한 신호**가 된다는 점(5.2절)에 착안해, 모델의 예측 결과를 책의 평가·호불호·장단점으로 풀어 주는 웹 서비스를 Streamlit으로 구현했다(`sentiment_app.py`). 웹크롤링한 14만+ 건의 리뷰 분석 결과를 누구나 검색·탐색할 수 있는 형태로 제공한다.

9.1 다섯 개의 기능 탭

- ① **책 검색·분석** — 책 제목을 검색하면 리뷰 수·평균 평점·감성 분포(긍/중/부)와 함께, 리뷰에서 뽑은 '좋았던 점 / 아쉬운 점' 키워드와 호불호 지수, 감성별 리뷰 예시를 보여 준다.
- ② **랭킹·큐레이션** — 긍정 비율·호불호·리뷰 수·평균 평점 기준으로 추천 도서를 표로 제시한다(최소 리뷰 수 조절 가능).
- ③ **트렌드** — 카테고리별 긍정 비율, 연도별 감성 추이를 보여 준다.
- ④ **책 비교** — 두 책을 나란히 두고 평점·감성·키워드를 비교한다.

- ⑤ **리뷰 직접 분석** — 사용자가 새 리뷰를 입력하면 학습된 모델이 즉시 긍정/중립/부정을 예측한다(단일 입력과 CSV 일괄 처리 모두 지원).

9.2 사전 계산 인덱스로 빠른 응답

①~④ 데이터 탭은 매번 모델을 돌리지 않는다. 별도 배치 스크립트(**build_book_index.py**)가 예측이 끝난 리뷰 CSV를 책 단위로 미리 집계해 **insight_data.pkl**이라는 인덱스 파일을 만들어 두고, 앱은 이를 즉시 불러와 보여 준다. 책별 리뷰 수·감성 분포·평균 평점·호불호 지수·키워드·예시 리뷰가 모두 사전 계산되어 있어 응답이 빠르다. 모델은 ⑤ '리뷰 직접 분석' 탭을 쓸 때만 로딩된다.

9.3 키워드 추출 — 모델 약점을 우회한 설계

'좋았던 점 / 아쉬운 점' 키워드는 재학습 없이, **긍정 리뷰와 부정 리뷰의 명사 빈도를 대비해** 한쪽에서 두드러지는 단어를 뽑는 방식으로 만든다. 핵심은 그룹을 나누는 기준이다.

- 평점이 있는 **YES24는 고평점(8~10점)과 저평점(1~4점) 리뷰**로 그룹을 나눠 키워드를 추출한다. 따라서 이 키워드는 **모델 정확도와 무관하게** 실제 평점에 근거하여 신뢰도가 높다.
- 평점이 없는 Watcha는 모델 예측 감성으로 그룹을 나눈다.
- 호불호 지수는 긍정·부정 비율이 모두 높을수록 커지도록 정의해(두 비율 중 작은 값 × 2), 의견이 갈리는 책을 한눈에 드러낸다.

이는 7장에서 짚은 모델의 한계를 서비스 차원에서 영리하게 우회한 설계다. 즉 정확도가 낮은 개별 예측에 의존하는 대신, 가능한 곳에서는 실제 평점이라는 더 단단한 근거를 활용한다.

9.4 실행과 배포

앱 실행에 꼭 필요한 파일은 앱 본체와 모델 3종(model_3class/), 그리고 인덱스(insight_data.pkl)뿐이며, 수백 MB의 원본·결과 CSV는 인덱스를 다시 만들 때만 쓰인다. 의존성은 requirements.txt(파이썬 패키지)와 packages.txt(Okta용 Java)로 정리해, Streamlit Community Cloud에 그대로 올리면 자동 설치되어 배포되도록 구성했다. 리뷰 직접 분석 탭을 제외하면 모델·Java 없이 데이터 탭만으로도 가볍게 배포할 수 있다.

10. 맺음말

본 프로젝트는 성격이 다른 세 사이트에서 도서 리뷰를 대규모로 수집하고, 라벨 불균형·번역 비용·과적합 같은 현실적인 문제들을 하나씩 해결하며 한국어 3-class 감성 분석 모델을 완성한 뒤, 그 결과를 누구나 활용할 수 있는 도서 인사이트 서비스로 확장한 전 과정을 담고 있다. 개별 리뷰 분류 정확도(57%)는 사전학습 모델 도입 등으로 끌어올려야 할 과제로 남지만, '낮은 정확도의 예측이라도 대량으로 모으면 의미 있는 통계가 된다'는 점과 '가능한 곳에서는 더 단단한 근거(실제 평점)를 활용한다'는 설계 원칙을 통해, 모델의 한계 안에서도 쓸모 있는 서비스를 만들 수 있음을 확인했다. 향후 계획(8장)을 순차적으로 적용하면 분류 정확도와 서비스 품질을 함께 높일 수 있을 것으로 기대한다.